Ввод и вывод данных

Подтемы занятия

- стандартные потоки ввода/вывода
- файловые потоки
- ввод/вывод объектов пользовательских типов

Стандартные потоки ввода/вывода

Для ввода/вывода в C++ реализованы классы-потоки:

istream и ***ostream***

Класс-поток	***Объект***	***Назначение***
istream	cin	Ввод данных
ostream	cout	Буферизированный вывод
ostream	clog	Буферизированный вывод служебной информации
ostream	cerr	Небуферизированный вывод ошибок

Для их использования требуется директива: ***#include <iostream>***

Буферизация потоков

Как работает буферизация?

Программа	Буфер (память)	Устройство вывода
Инициирует вывод данных	Накапливает выводы данных в ОЗУ до превышения лимита	Выводит данные на экран, в файл и т.п.

Обращение к устройству вывода – *медленная операция*. Для того, чтобы их минимизировать, используется ***буфер***.

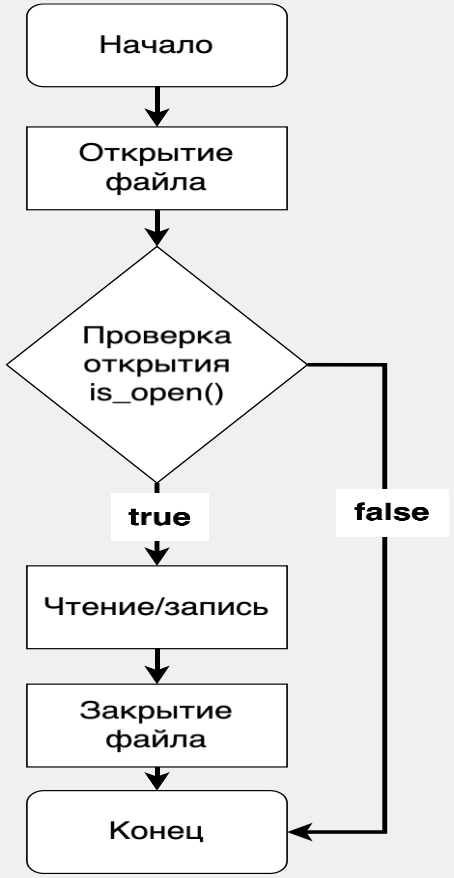
Когда в буфере накопляется достаточное количество сообщений, его содержимое сбрасывается и выводится на экран.

Файловые потоки

Для работы с файлами предусмотрены 3 класса:

Класс	***Назначение***
ifstream	Чтение данных из файла
ofstream	Запись данных в файл
fstream	Чтение и запись одновременно

Алгоритм работы с файлами:



Открытие файлов

Способы открытия файлов

1. Через конструктор

```
std::ofstream ofs("file.txt");
```

2. Через метод *open()*

```
std::ofstream ofs;
ofs.open("file.txt");
```

Пути к файлам

1. Относительный путь (относительно exe файла)

C:\Student\Project

- **program.exe**
- **data.txt <- “data.txt”**

2. Абсолютный путь (полный)

C:\Student\Project\data.txt

При открытии файлов на запись (out) его содержимое ***удаляется***!

Режимы открытия файлов

Методы open() и конструкторы принимают режимы:

Режим	*Назначение*
ios::in	Открыть для чтения
ios::out	Открыть для записи (очищает файл)
ios::app	Дозапись в конец (старые данные сохранены)
ios::ate	Указатель в конец при открытии
ios::trunc	Усечь файл (если существует)
ios::binary	Бинарный режим

Режимы открытия файла можно комбинировать через “|”:

```
std::fstream file("data.txt", std::ios::in | std::ios::ate);
```

Навигация по файлу

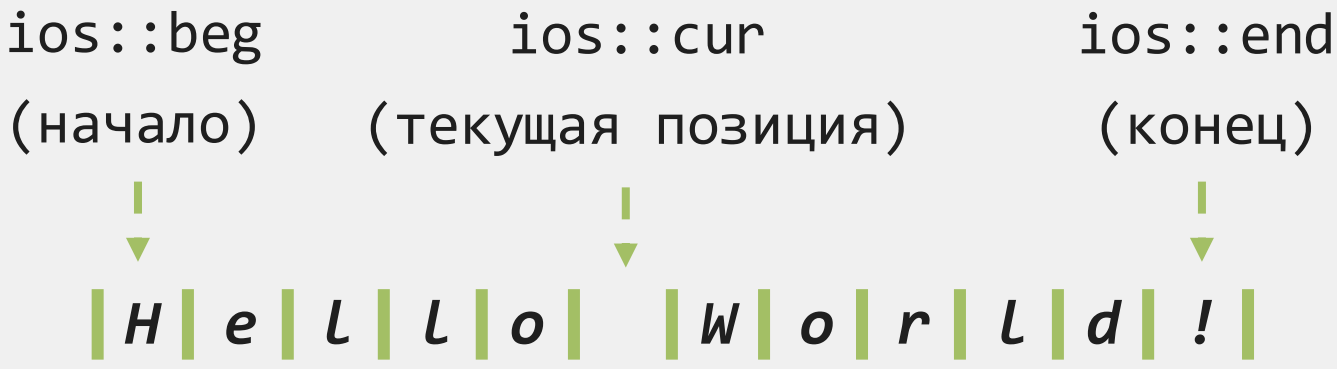
Работа с файлом происходит в формате «считать/записать N байт».

Данные действия выполняются относительно ***указателя*** в файле.

Для смещения указателя используются функции:

- seekg(offset, origin) - указатель чтения (get)
- seekp(offset, origin) - указатель записи (put)

origin – относительно чего смещать:



Ввод-вывод своих типов

Задача: Вывести объект класса *Person* так же легко, как *int*

```
int x = 5;
std::cout << x; // ОК, встроенный тип.

Person p("Иван", 20);
std::cout << p; // ОШИБКА КОМПИЛЯЦИИ! cout не знает про Person.
```

Наивное решение – создать метод ***print()*** в классе *Person*.

```
p.print(); // Работает, но не совместимо с потоками.
```

Для единообразия необходимо перегрузить операторы << и >>

Перегрузка операторов << и >>

Как правильно перегружать?

Вызов	***Сигнатура функции***
`cout << p`	`ostream& operator<<(ostream& os, const Person& p)`
`cin >> p`	`istream& operator>>(istream& is, Person& p)`

Ключевые правила:

- функция должна быть внешней
- функция должны быть объявлена *friend*
- функция должна возвращать ссылку на поток

Цепочка вызовов:

```
cout << p1 << p2
```

```
operator<<(operator<<(cout, p1), p2)
```

Конец презентации

Разработчик

Марышев А.В.

Контакты для связи

Телеграм: @XekS_s